Introduction to Generative AI

Dr. Mainack Mondal
IIT Kharagpur

SESSION ORIENTATION

By the end of this session, you will be able to...

1

Explain

why raw text must be tokenized and embedded before any model can process it

2

Derive

the maths behind subword tokenization, embeddings, and positional encoding

3

Compute

a self-attention pass by hand on a toy example, step by step

4

Implement

minimal, working Python for each building block covered today

SESSION FLOW

Today's Roadmap

01	NLP Foundations: NLP, NLU & Tokens
02	Why Generative AI, Why Now
03	Generative Model Families: Seq2Seq, VAEs & Friends
04	Tokenizers
05	Embeddings
06	Transformer Architecture & Positional Encoding
07	Self-Attention Deep Dive
08	Wrap-up & Takeaways

We will return to this roadmap between every module.

SESSION FLOW

Module 01 of 08

01	NLP Foundations: NLP, NLU & Tokens
02	Why Generative AI, Why Now
03	**Generative Model Families: Seq2Seq, VAEs & Friends**
04	Tokenizers
05	Embeddings
06	Transformer Architecture & Positional Encoding
07	Self-Attention Deep Dive
08	Wrap-up & Takeaways

What Is NLP?

- Natural Language Processing (NLP) is the field of AI concerned with getting computers to work with human language — text and speech.
- It is an umbrella term covering everything from spell-checkers to chatbots to generative AI models.
- “Working with language” splits into two directions:
- • taking language IN and making sense of it (understanding)
- • producing language OUT (generation)

Everyday NLP, already around you

- Spam filters in your inbox
- Autocomplete & spell-check
- Search engines
- Voice assistants (Siri, Alexa)
- Translation apps
- Chatbots & AI assistants

NLP vs. NLU vs. NLG

NLP

Natural Language Processing

The umbrella field: any computational work with human language.

NLU

Natural Language Understanding

Making sense of language — intent, meaning, entities, sentiment.

NLG

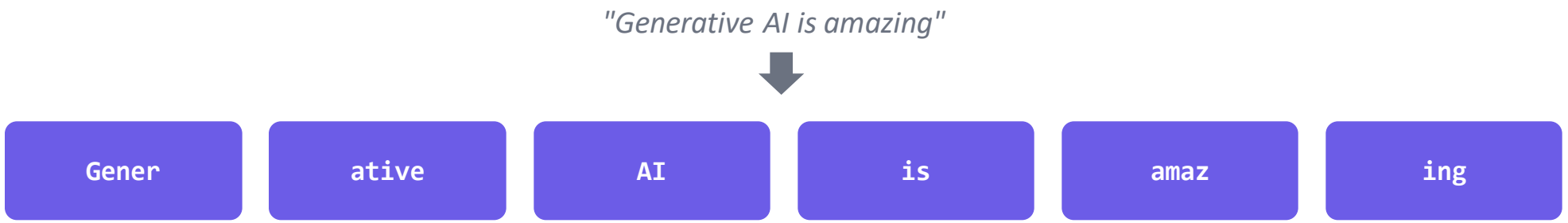
Natural Language Generation

Producing fluent language — answers, summaries, translations.

Generative AI models like GPT do both: they understand a prompt (NLU) and generate a response (NLG).

What Is a Token?

- Computers cannot read words directly — everything must first become numbers.
- **A token is the small chunk of text a model treats as one unit — sometimes a whole word, sometimes a piece of one, sometimes a single character.**
- Splitting text into tokens is called tokenization — it is always the very first step before any NLP model can process language.
- We'll go much deeper into exactly how this splitting works in Module 04.



6 tokens → each will be converted to a number (an ID), then to a vector — that's next in Modules 03–04.

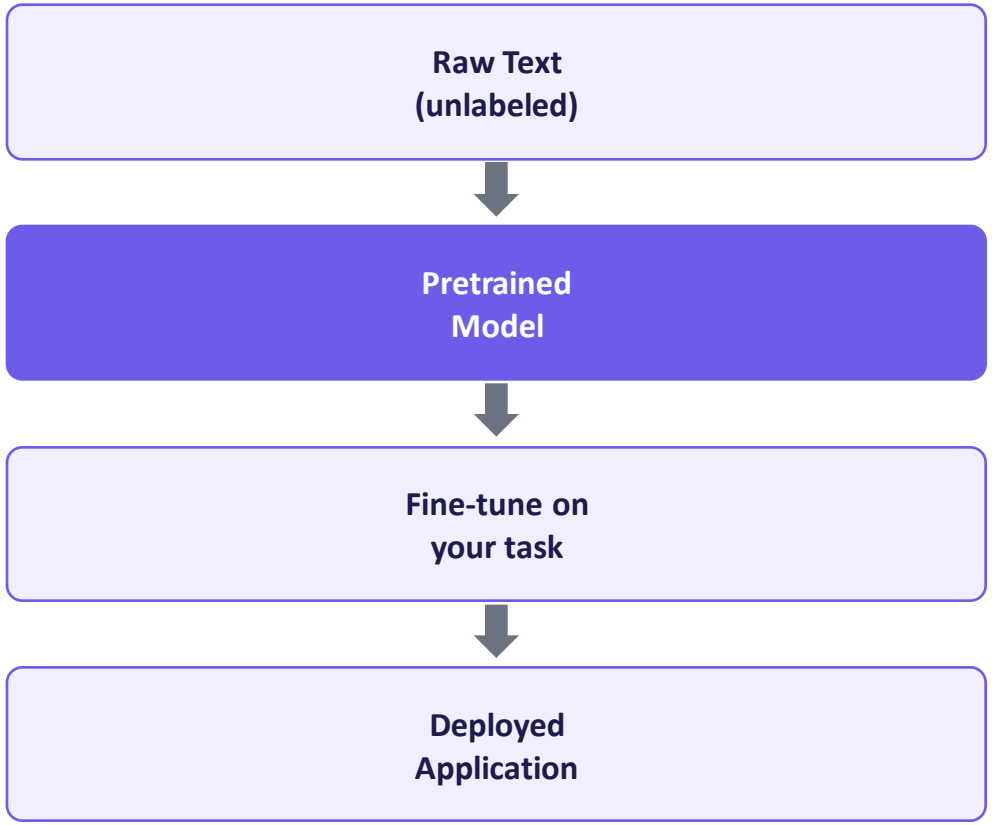
SESSION FLOW

Module 02 of 08

01	NLP Foundations: NLP, NLU & Tokens
02	Why Generative AI, Why Now
03	**Generative Model Families: Seq2Seq, VAEs & Friends**
04	Tokenizers
05	Embeddings
06	Transformer Architecture & Positional Encoding
07	Self-Attention Deep Dive
08	Wrap-up & Takeaways

The Transfer-Learning Shift

- Pre-2018: every NLP task needed its own labeled dataset and a model trained from random weights.
- 2018 onward: pretrain one large model on raw text (self-supervised), then fine-tune it cheaply on any downstream task.
- Text on the open web already encodes grammar, world knowledge, and reasoning patterns — pretraining is how a model absorbs it.
- **Generative AI = this pretrained model used directly to produce text, code, images, or audio.**



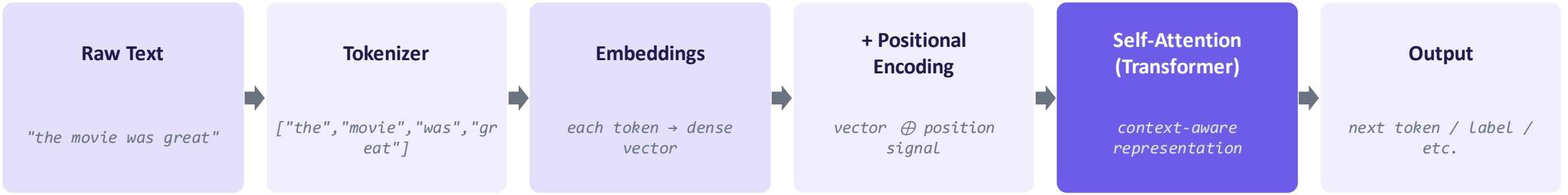
Concept: Pretrain-then-Finetune paradigm (2018–)

How We Got Here: A Compressed Timeline

1950s–80s	**Rule-based NLP**	Hand-written grammars and symbolic logic
1990s–2000s	**Statistical NLP**	n-grams, HMMs, count-based methods
2013	**Word2Vec / GloVe**	Dense word embeddings from co-occurrence
2014–16	**RNNs / LSTMs + Attention**	Sequential models, seq2seq with attention
2017	**“Attention Is All You Need”**	The Transformer architecture
2018–today	**BERT, GPT, LLMs**	Pretrain-then-finetune / prompt at scale

Adapted from Jurafsky & Martin, SLP3; Vaswani et al. 2017

The Approximate Flow We'll Trace Today



Today's four building blocks — tokenizer, embeddings, positional encoding, self-attention — are the machinery inside every step above.

Note: This is an *approximate* flow for building intuition — production transformers stack many attention + feed-forward layers, use residual connections and layer norm, and process all tokens in parallel rather than one at a time.

SESSION FLOW

Module 03 of 08

01	NLP Foundations: NLP, NLU & Tokens
02	Why Generative AI, Why Now
03	**Generative Model Families: Seq2Seq, VAEs & Friends**
04	Tokenizers
05	Embeddings
06	Transformer Architecture & Positional Encoding
07	Self-Attention Deep Dive
08	Wrap-up & Takeaways

A Quick Tour of Generative Model Families

Before autoregressive transformers took over, researchers tried several very different ways to make machines create something new.

2013	**Variational Autoencoders**	Learn a probability distribution you can sample from
2014	**Seq2Seq (Sutskever et al.)**	Encoder-decoder RNNs — read it all, then write it all
2014	**GANs (Goodfellow et al.)**	Two networks compete until the forger wins
2015–17	**Attention, then Transformers**	Fix Seq2Seq's bottleneck — covered in Modules 06–07
2015 / 2020+	**Diffusion Models**	Sculpt a clean output out of pure noise, step by step

Same question, five different answers: "how do you get a model to produce something new?"

Sutskever et al. 2014; Kingma & Welling 2013; Goodfellow et al. 2014; Sohl-Dickstein et al. 2015

First, the Pattern Behind (Almost) Everything

- Nearly every architecture in this module — and a large share of deep learning generally — is built from two cooperating pieces: an Encoder and a Decoder.
- **Encoder — reads the raw input and compresses it into a compact internal representation: a vector (or set of vectors) that captures "what the input means."**
- **Decoder — takes that internal representation and produces an output: reconstructing it, translating it, or generating something brand new from it.**
- Keep this shape in mind — you'll see it again in Seq2Seq, in Autoencoders and VAEs, and later even inside the Transformer itself.



Seq2Seq: The Pattern, Applied to Sequences

- Seq2Seq is the **encoder-decoder** pattern built from two RNNs:
  - an Encoder reads the input one token at a time; a Decoder writes the output one token at a time.
- **The encoder's final hidden state becomes the “representation” from the last slide — here called a “context vector” — a compressed summary of the entire input sentence.**
- The decoder is conditioned on that context vector (plus whatever it has already generated) to produce each output token.
- Classic use case: machine translation.



Sutskever, Vinyals & Le, 2014, “Sequence to Sequence Learning with Neural Networks”

Worked Example: One Word at a Time

Encode	`h1=f("I")  h2=f("am",h1)  h3=f("a",h2)  h4=f("student",h3)`	*Final state h4 = context vector c*
Decode 1	`input: <START> + c  → output: “Je”`	*First decoder step*
Decode 2	`input: “Je” + c  → output: “suis”`	*Feed previous output back in*
Decode 3	`input: “suis” + c  → output: “étudiant”`	*Keep going…*
Decode 4	`input: “étudiant” + c  → output: <END>`	*Stop token → translation complete*

Notice: the decoder never looks back at the original sentence directly — only at the one compressed vector c. Keep that in mind for the next slide…

The Maths: Chain Rule Over Output Tokens

$$P(y_1,\ldots,y^T \mid x_1,\ldots,x^S) = \prod_{t=1}^{T} P(y_t \mid y_1,\ldots,y_{t-1}, c)$$

- The whole output sentence's probability factorizes token by token — each token depends on everything generated so far, plus the fixed context c.
- Training uses “teacher forcing”: feed the true previous token, not the model's own (possibly wrong) guess, to keep learning stable.
- At inference time, decoding is greedy (pick the top token each step) or beam search (track several candidate sequences at once).

The Catch: One Vector Has to Carry Everything

- **Squeezing an entire sentence — or paragraph — into one fixed-size vector is a serious bottleneck.**
- Long inputs lose detail: by the time the encoder reaches the last word, early words have faded from memory.
- The fix: attention (Bahdanau et al., 2015) — let the decoder look back at ALL encoder states, not just the last one.
- **Self-attention (Module 07) takes that idea and applies it everywhere, all at once, with no recurrence at all.**



Bahdanau, Cho & Bengio, 2015, "Neural Machine Translation by Jointly Learning to Align and Translate"

Code: A Minimal Encoder-Decoder

```
import torch.nn as nn

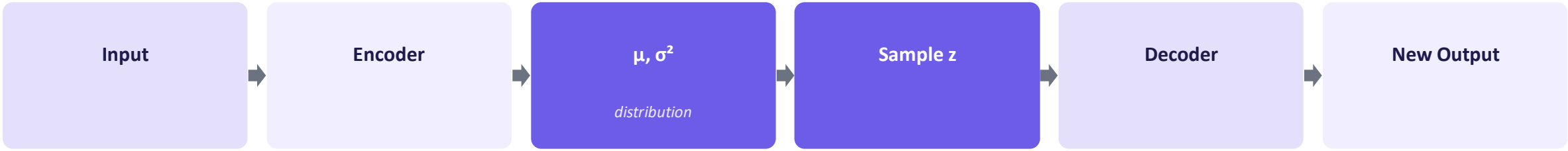
class Encoder(nn.Module):
    def __init__(self, vocab_size, hidden_size):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, hidden_size)
        self.rnn = nn.GRU(hidden_size, hidden_size, batch_first=True)
    def forward(self, x):
        _, context = self.rnn(self.embed(x))   # context = final hidden state
        return context

class Decoder(nn.Module):
    def forward(self, y_prev, context):
        out, context = self.rnn(self.embed(y_prev), context)
        return self.fc_out(out), context        # predict next token
```

Simplified sketch — omits embedding/output-layer setup for brevity

Compress, Then Sample

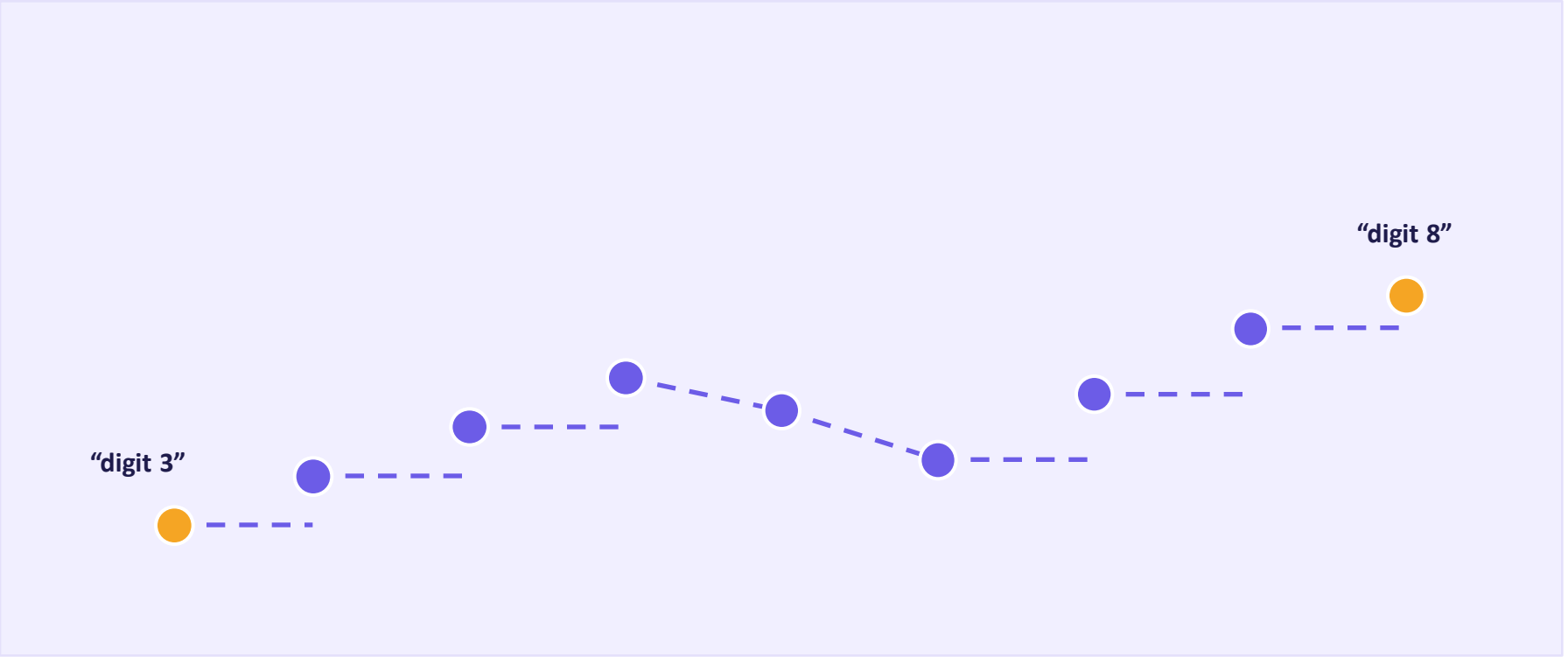
- An Autoencoder applies the encoder-decoder pattern to compression: an Encoder squeezes input into a small "latent code," a Decoder reconstructs it — trained just to minimize reconstruction error, no labels needed.
- **A Variational Autoencoder (VAE) adds one twist: instead of encoding to one exact point, it encodes to a probability distribution (a mean and a variance).**
- **That twist is what makes it generative — sample a new random point from the distribution, decode it, and you get a plausible, brand-new output instead of a copy of training data.**
- Less central to today's state-of-the-art image/text generation than a few years ago, but the "compress into a distribution, then sample" idea still echoes across the field.



Kingma & Welling, 2013, "Auto-Encoding Variational Bayes"

One Example: Strolling Through Latent Space

Illustrative 2D latent space — nearby points decode to visually similar (but new) outputs:



Every point along the dashed path is a valid sample — walk from one region to another and the output morphs smoothly. On to the families still driving today's frontier models.

Classic illustration style from Kingma & Welling-style VAE demos on MNIST digits

Two More Tricks: Adversaries and Denoising

GANs

- A Generator creates fake samples from random noise.
- A Discriminator tries to tell real from fake.
- They train against each other — an adversarial arms race — until the generator produces samples the discriminator can't catch.
- **Great at sharp, high-resolution images.**

Diffusion

- Start from pure random noise.
- Train a model to gradually remove a little noise at each step.
- After many small denoising steps, structured, coherent output emerges from static.
- **Powers most of today's state-of-the-art image generators.**

Goodfellow et al. 2014; Sohl-Dickstein et al. 2015; Ho et al. 2020 (DDPM)

Same Goal, Five Different Playbooks

Family	Core Idea	Great For
Autoregressive (GPT-style)	Predict the next token given all previous ones	Open-ended text generation, chat
Seq2Seq	Encode input fully, then decode conditioned on it	Translation, summarization
VAE	Learn a samplable probability distribution	Smooth latent exploration, anomaly detection
GAN	Generator vs. discriminator, adversarial training	Sharp, high-resolution images
Diffusion	Iteratively denoise from pure noise	Today's top image / audio generators

Spoiler: the rest of today's lecture is about the family that won for language — autoregressive transformers.

Generative Model Families: What to Remember

- ✓ Nearly every family in this module shares one shape: an Encoder that compresses input, and a Decoder that produces output from that compression.
- ✓ Seq2Seq applies that pattern with RNNs — and exposed the context-vector bottleneck that attention was invented to solve.
- ✓ VAEs generate by sampling from a learned, continuous probability distribution rather than looking up training data.
- ✓ Today's frontier LLMs are autoregressive transformers — understanding the alternatives helps explain why that architecture won for language.

Spot the Family

Q1

A model that starts from random static and gradually “sculpts” a photorealistic face — which family is this?

ANSWER

Diffusion — iterative denoising from pure noise, step by step.

Q2

A model trained by pitting a “forger” network against a “detective” network until the forgeries are undetectable — which family?

ANSWER

GAN (Generative Adversarial Network) — generator vs. discriminator, in competition.

SESSION FLOW

Module 04 of 08

01	NLP Foundations: NLP, NLU & Tokens
02	Why Generative AI, Why Now
03	Generative Model Families: Seq2Seq, VAEs & Friends
04	**Tokenizers**
05	Embeddings
06	Transformer Architecture & Positional Encoding
07	Self-Attention Deep Dive
08	Wrap-up & Takeaways

History: Deciding What Counts as a “Unit”

- Word-level tokenization (pre-2015): split on whitespace/punctuation.
- Problem: huge vocabularies, and any unseen word becomes an unusable “UNK” token.
- Character-level: tiny vocabulary, no unseen-word problem — but sequences become very long and hard to model.
- **Byte-Pair Encoding, BPE (Sennrich et al., 2016): borrowed from data compression (Gage, 1994) — the sweet spot.**
- Descendants in production today: WordPiece (BERT), BPE (GPT family), SentencePiece / Unigram (T5, LLaMA).

Sennrich et al. 2016, “Neural Machine Translation of Rare Words with Subword Units”

Technique: Subword Tokenization

- A tokenizer maps a text string to a sequence of integer IDs from a fixed vocabulary.
- Subword methods start from characters and merge the most frequent adjacent pairs, so:
- • common words stay whole (“the”, “running”)
- • rare/unseen words split into familiar pieces (“unhappiness” → “un” + “happi” + “ness”)
- Vocabulary size is a hyperparameter — typically 30K–100K+ tokens for modern LLMs.

Granularity	Vocab size	Sequence length	Unseen words
Character	~100	Very long	Never unknown
Subword (BPE)	30K–100K	Moderate	Rare, graceful
Word	100K+	Short	Frequent “UNK”

Worked Example: Byte-Pair Encoding (BPE) by Hand

Toy corpus (word: frequency): low: 5, lower: 2, newest: 6, widest: 3

Step 0	`l o w · (5)  l o w e r · (2)  n e w e s t · (6)  w i d e s t · (3)`	*Start: every word split into characters + end marker ·*
Step 1	`Most frequent pair: (e, s) → 9 occurrences`	*Merge → new symbol “es”*
Step 2	`Next most frequent pair: (es, t) → 9 occurrences`	*Merge → new symbol “est”*
Step 3	`Next: (l, o) → 7 occurrences`	*Merge → new symbol “lo”*
Step 4	`Continue until vocabulary budget is reached…`	*Final vocab includes: “low”, “low”+“er”, “new”+“est”, “wid”+“est”*

Simplified illustrative pass; production BPE runs thousands of merges

The Maths: BPE as Greedy Merge Selection

$$\text{pair}^* = \operatorname{argmax}_{(a,b)} \quad \text{count}(a, b) \quad \text{s.t. } (a,b) \text{ are adjacent symbols across the corpus}$$

Merge pair into a single new symbol; repeat for a fixed number of merges (the vocabulary-size budget).*

- Every merge step is deterministic and greedy — always take the single most frequent adjacent pair.
- Number of merges = final vocabulary size − initial character vocabulary size.
- Encoding a new word re-applies the learned merges in the order they were learned.
- Complexity: O(corpus length) per merge with a priority queue — tractable even at web scale.

Code: Tokenizing With a Pretrained Tokenizer

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("gpt2")

text = "Generative AI tokenizes unfamiliar words into pieces."
ids = tokenizer.encode(text)
tokens = tokenizer.convert_ids_to_tokens(ids)

print(tokens)
# ['Gener', 'ative', 'ĠAI', 'Ġtoken', 'izes', 'Ġunfamiliar', ...]
print(len(tokens), "tokens for", len(text), "characters")
```

Note: Ġ marks a preceding space in GPT-2's byte-level BPE vocabulary.

Tokenizers: What to Remember

- ✓ Tokenization converts raw text into a finite vocabulary of integer IDs — the very first step of any pipeline.
- ✓ Subword methods (BPE, WordPiece, SentencePiece) balance vocabulary size against sequence length.
- ✓ The greedy “merge the most frequent pair” rule is simple, fast, and remarkably effective at scale.
- ✓ Tokenization choices affect cost, context-window usage, and how well rare words / languages are handled.

Before We Move On…

Q1

Why can't a model just read raw characters one at a time and call it a day?

ANSWER

It could — but sequences would explode in length. Subwords hit the sweet spot between vocabulary size and sequence length.

Q2

“Unbelievably” might tokenize as [“Un”, “believ”, “ably”]. Why split it at all if the model already “knows” the word?

ANSWER

It probably doesn't! Rare/long words are often unseen during training — splitting into familiar pieces lets the model handle them anyway.

SESSION FLOW

Module 05 of 08

01	NLP Foundations: NLP, NLU & Tokens
02	Why Generative AI, Why Now
03	Generative Model Families: Seq2Seq, VAEs & Friends
04	Tokenizers
05	**Embeddings**
06	Transformer Architecture & Positional Encoding
07	Self-Attention Deep Dive
08	Wrap-up & Takeaways

From Token IDs to Meaning

- Module 04 turned text into integer IDs — but an ID is just an arbitrary label, not meaning.
- “cat” → 4521 and “dog” → 892: nothing about those two numbers tells you the words are related.
- Token 4522 (whatever it happens to be) isn't necessarily close in meaning to “cat” just because it's next on the vocabulary list.
- **What we need: a representation where similar concepts sit near each other, and relationships between words become directions we can compute with. That representation is an embedding.**

Token IDs

`cat → 4521` `dog → 892` `amazing → 77`

Arbitrary numbers — no notion of similarity.



Embeddings

`cat → [0.62, 0.91, …]` `dog → [0.58, 0.85, …]`

Similar meaning → nearby vectors.

History: From One-Hot to Contextual Vectors

Pre-2013	**One-hot encoding**	Sparse, huge vectors; no notion of similarity
2013	**Word2Vec (Mikolov et al.)**	Dense vectors learned by predicting neighbors
2014	**GloVe (Pennington et al.)**	Global co-occurrence statistics → vectors
2018+	**Contextual embeddings (ELMo, BERT)**	Same word → different vector per context

Technique: Meaning as Geometry

- Each token ID is mapped to a dense vector (e.g. 768 or 4096 real numbers) via a learned lookup table.
- Vectors are learned so that words appearing in similar contexts end up close together in space.
- Distance/angle between vectors becomes a proxy for semantic similarity.
- **Everything downstream — attention, generation — operates on these vectors, not on the original text.**



Illustrative 2D projection of a high-dimensional embedding space

Worked Example: king – man + woman ≈ queen

```
v(king) – v(man) + v(woman)  ≈  v(queen)
```

- Toy 4-dimensional embeddings (illustrative, not real trained values):

king	`[0.62, 0.91, 0.15, 0.44]`
man	`[0.55, 0.20, 0.10, 0.30]`
woman	`[0.58, 0.85, 0.12, 0.33]`
king – man + woman	`[0.65, 1.56, 0.17, 0.47]`
queen (nearest neighbor)	`[0.66, 1.50, 0.18, 0.50]`

The Maths: Cosine Similarity & the Skip-Gram Objective

$$\cos(u, v) = (u \cdot v) / (\|u\| \, \|v\|)$$

Ranges from −1 (opposite) to 1 (identical direction) — the standard similarity score between two embeddings.

$$P(o|c) = \text{softmax}(v_o \cdot v_c)$$

Skip-gram trains embeddings by predicting nearby words — words used similarly end up with similar vectors.

- Training signal needs no human labels — the text itself supplies “correct answers” (self-supervised learning).
- Embedding dimensionality is a design choice: bigger → more expressive, but more parameters and compute.
- In modern transformers, the embedding table is simply the first learned weight matrix of the network.

Mikolov et al. 2013, “Efficient Estimation of Word Representations”

Code: Cosine Similarity Between Embeddings

```
import numpy as np

def cosine_similarity(u, v):
    return np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))

king  = np.array([0.62, 0.91, 0.15, 0.44])
queen = np.array([0.66, 1.50, 0.18, 0.50])
apple = np.array([0.10, 0.05, 0.92, 0.20])

print(cosine_similarity(king, queen))  # ~0.98  -> close in meaning
print(cosine_similarity(king, apple))  # ~0.35  -> unrelated
```

Embeddings: What to Remember

- ✓ Embeddings turn discrete token IDs into continuous vectors that capture meaning.
- ✓ Similarity in vector space (cosine similarity) approximates semantic similarity in language.
- ✓ Embeddings are learned end-to-end, either via objectives like skip-gram or jointly inside a transformer.
- ✓ They are the input representation that every later transformer layer builds on.

Vector Space Trivia

Q1

If v(Paris) − v(France) + v(Japan) pointed somewhere sensible, what word would you expect to land near?

ANSWER

“Tokyo” — the same capital-of relationship, transplanted onto a different country vector.

Q2

Two totally different words could have very similar embeddings. When might that happen?

ANSWER

When they're used in near-identical contexts — e.g. “excellent” and “outstanding.” Embeddings capture usage, not dictionary definitions.

SESSION FLOW

Module 06 of 08

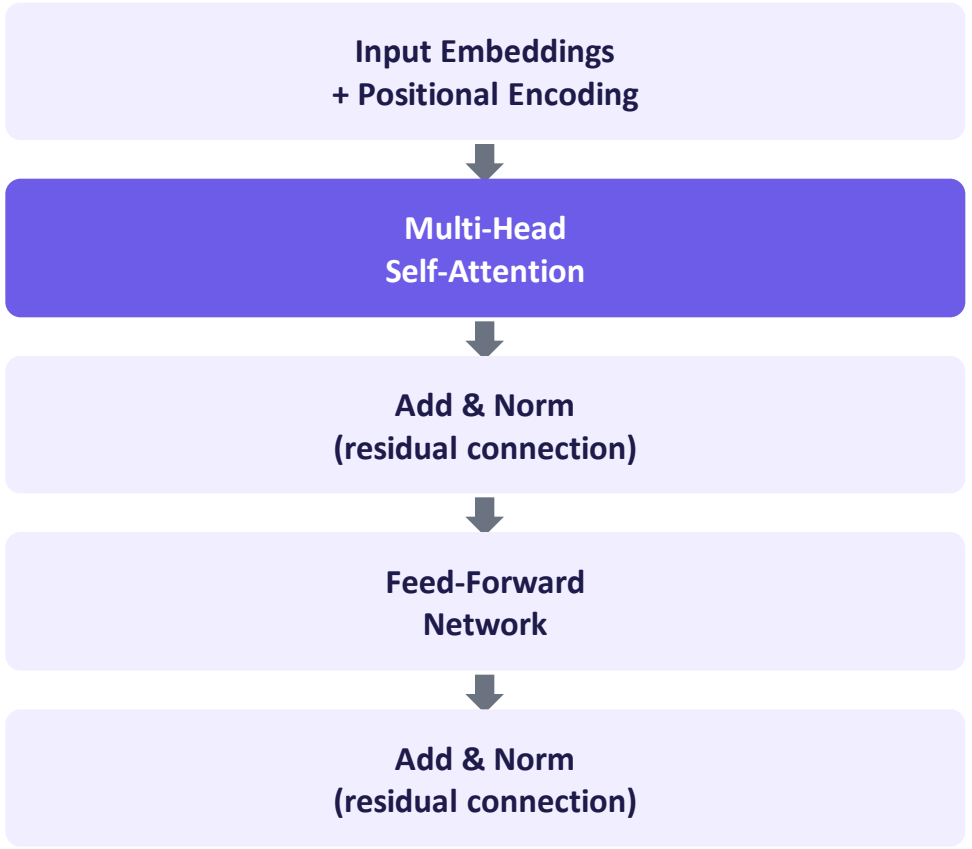
01	NLP Foundations: NLP, NLU & Tokens
02	Why Generative AI, Why Now
03	Generative Model Families: Seq2Seq, VAEs & Friends
04	Tokenizers
05	Embeddings
06	**Transformer Architecture & Positional Encoding**
07	Self-Attention Deep Dive
08	Wrap-up & Takeaways

History: Why Attention Replaced Recurrence

- RNNs/LSTMs process tokens one at a time, in order — hard to parallelize, and long-range dependencies fade.
- 2014–15: attention mechanisms added to seq2seq models let the decoder "look back" at all encoder states.
- **2017: Vaswani et al. publish "Attention Is All You Need" — drop recurrence entirely, use only attention.**
- Result: full parallelism during training, better long-range modeling, and the architecture behind BERT, GPT, T5, and virtually every modern LLM.

Vaswani et al. 2017, "Attention Is All You Need"

Technique: Anatomy of a Transformer Block



This block repeats N times (e.g. N = 12–96+ layers) — stacking lets the model build increasingly abstract representations of the input.

Simplified encoder block; decoder adds masked + cross-attention

Why Position Must Be Injected Explicitly

- **Self-attention treats input as an unordered set — it is permutation-invariant by construction.**
- “The dog bit the man” and “The man bit the dog” would look identical to raw self-attention without position info.
- Solution: add a position-dependent vector to each token embedding before the first attention layer.
- The encoding must be unique per position and let the model infer relative distances between tokens.

token embedding + positional encoding = input to the first attention layer

The Maths: Sinusoidal Position Vectors

$$PE(pos, 2i) = \sin\left(pos / 10000^{(2i / d_model)} \right)$$

$$PE(pos, 2i+1) = \cos\left(pos / 10000^{(2i / d_model)} \right)$$

- pos = token position in the sequence (0, 1, 2, …); i = dimension index within the embedding.
- Each dimension oscillates at a different frequency — low i → fast oscillation, high i → slow oscillation.
- Because sin/cos are smooth and periodic, the model can learn to attend by relative offset (“3 tokens back”), not just absolute position.

Vaswani et al. 2017, Section 3.5

Worked Example: PE Values for d_model = 4

Using PE(pos,2i)=sin(pos/10000^(2i/4)), PE(pos,2i+1)=cos(pos/10000^(2i/4))

pos	dim0 (sin)	dim1 (cos)	dim2 (sin)	dim3 (cos)
0	0.000	1.000	0.000	1.000
1	0.841	0.540	0.010	1.000
2	0.909	-0.416	0.020	1.000
3	0.141	-0.990	0.030	1.000

Each row is a unique 4-number “fingerprint” added to that position's token embedding.

Code: Computing Sinusoidal Encodings

```
import numpy as np

def positional_encoding(seq_len, d_model):
    pos = np.arange(seq_len)[:, None]
    i   = np.arange(d_model)[None, :]
    angle_rates = 1 / np.power(10000, (2 * (i // 2)) / d_model)
    angles = pos * angle_rates
    pe = np.zeros_like(angles)
    pe[:, 0::2] = np.sin(angles[:, 0::2])   # even dims
    pe[:, 1::2] = np.cos(angles[:, 1::2])   # odd dims
    return pe

pe = positional_encoding(seq_len=4, d_model=4)
```

Transformer Architecture: What to Remember

- ✓ Transformers replaced recurrence with attention, enabling parallel training and better long-range context.
- ✓ A transformer block = self-attention + feed-forward, each wrapped with residual connections and normalization.
- ✓ Positional encoding restores the notion of word order that self-attention alone discards.
- ✓ Stacking many identical blocks lets the model build progressively richer representations.

Swap the Words, Break the Sentence?

Q1

“Dog bites man” vs. “Man bites dog” — same words, same bag, wildly different meaning. What single mechanism fixes this for self-attention?

ANSWER

Positional encoding — injecting a position-dependent signal so order isn't lost the moment attention looks at the whole sentence at once.

SESSION FLOW

Module 07 of 08

01	NLP Foundations: NLP, NLU & Tokens
02	Why Generative AI, Why Now
03	Generative Model Families: Seq2Seq, VAEs & Friends
04	Tokenizers
05	Embeddings
06	Transformer Architecture & Positional Encoding
▶ 07	**Self-Attention Deep Dive**
08	Wrap-up & Takeaways

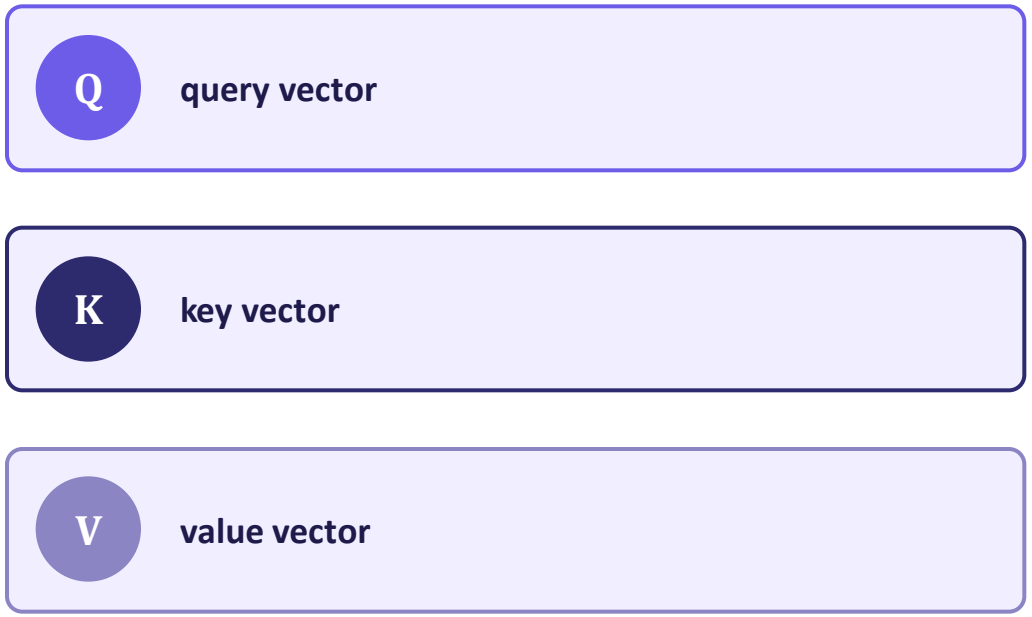
History: From “Look Back” to “Look at Everything, Weighted”

- Bahdanau et al. (2015): decoder attention lets a translation model softly align output words to relevant input words.
- **Self-attention (2017): apply the same idea within a single sequence — every token attends to every other token, including itself.**
- The output for each token becomes a weighted blend of all tokens' information, where the weights are learned and input-dependent.
- This is the mechanism that lets “bank” in “river bank” vs. “bank account” end up with different contextual meaning.

Bahdanau et al. 2015; Vaswani et al. 2017

Technique: Query, Key, Value

- Every token's embedding is projected into three vectors via learned weight matrices:
- **Query (Q) — “what am I looking for?”**
- **Key (K) — “what do I contain / advertise?”**
- **Value (V) — “what information do I actually pass on?”**
- A token's query is compared against every token's key to produce attention weights, which then blend the values.



The Maths: Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(QK^T / \sqrt{d_k} \right) V$$

- QK^T — dot product between every query and every key → raw similarity scores.
- $\sqrt{d_k}$ — scale down by the square root of the key dimension, to keep gradients stable.
- softmax(·) — turn scores into positive weights that sum to 1 across each row.
- · V — use those weights to take a weighted average of the value vectors.

Vaswani et al. 2017, Section 3.2.1

Worked Example: Attention Weights, Step by Step

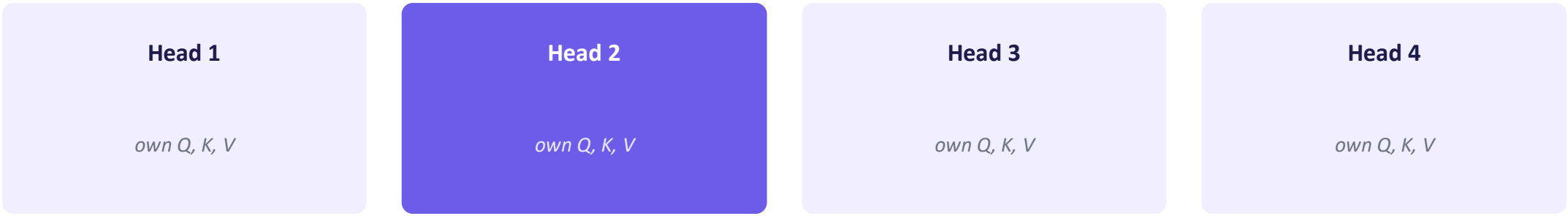
Two tokens, d_k = 2. Query and Key vectors for token "A" (attending) and token "B":

1	`q_A = [1, 0]  k_A = [1, 0]  k_B = [0, 1]`	*Given vectors*
2	`score_AA = q_A · k_A = 1  score_AB = q_A · k_B = 0`	*Raw dot-product scores*
3	`scaled: 1/√2 = 0.707  0/√2 = 0.000`	*Divide by √d_k = √2*
4	`softmax([0.707, 0.000]) = [0.670, 0.330]`	*Normalize into weights*
5	`output_A = 0.670·v_A + 0.330·v_B`	*Weighted blend of Values*

Token A ends up mostly representing itself (67%) with a smaller contribution from token B (33%) — exactly what "attending" means.

Extending It: Multi-Head Attention

- Running one attention function only lets the model focus on one kind of relationship at a time.
- **Multi-head attention runs several attention operations in parallel, each with its own learned Q/K/V projections.**
- One head might learn syntactic relationships (subject → verb), another might track coreference (pronoun → noun).
- Head outputs are concatenated and passed through one more learned projection to combine them.



Typical models use 8–96 heads per layer

Code: Scaled Dot-Product Attention in NumPy

```
import numpy as np

def softmax(x, axis=-1):
    e = np.exp(x - np.max(x, axis=axis, keepdims=True))
    return e / e.sum(axis=axis, keepdims=True)

def self_attention(Q, K, V):
    d_k = Q.shape[-1]
    scores  = Q @ K.T / np.sqrt(d_k)      # similarity, scaled
    weights = softmax(scores)             # rows sum to 1
    return weights @ V, weights           # weighted blend of V

output, attn_weights = self_attention(Q, K, V)
```

Self-Attention: What to Remember

- ✓ Self-attention lets every token gather information from every other token, weighted by learned relevance.
- ✓ Query, Key, and Value are three learned projections of the same input embedding, each with a distinct role.
- ✓ softmax(QK^T/√d_k)V is the entire mechanism — simple to state, powerful in effect.
- ✓ Multi-head attention runs this in parallel “views” so different heads can specialize in different relationships.

One Last Puzzle

Q1

In “The trophy didn't fit in the suitcase because it was too big,” what does “it” refer to — and how would self-attention figure that out?

ANSWER

“It” = the trophy. The word “it” attends strongly to “trophy” because context (“too big”) makes that the most relevant token to pull information from.

SESSION FLOW

Module 08 of 08

01	NLP Foundations: NLP, NLU & Tokens
02	Why Generative AI, Why Now
03	Generative Model Families: Seq2Seq, VAEs & Friends
04	Tokenizers
05	Embeddings
06	Transformer Architecture & Positional Encoding
07	Self-Attention Deep Dive
▶ 08	**Wrap-up & Takeaways**

WRAP-UP

Key Takeaways — All Four Building Blocks

Tokenizers

Text → subword units via greedy frequency-based merging (BPE).

Embeddings

Discrete tokens → dense vectors that encode meaning as geometry.

Positional Encoding

Sinusoidal signals restore order information that attention discards.

Self-Attention

softmax(QK^T/√d_k)V lets every token weigh every other token's relevance.